

DISTRIBUTED FILE SYSTEM (DFS)

Distributed File System (DFS)

- ▶ It is a file system that is distributed on multiple file servers or multiple locations.
- ▶ It allows programs to access or store isolated files as they do with the local ones, allowing programmers to access files from any network or computer..
- ▶ Benefits of DFS:
 - ▶ Remote information sharing
 - ▶ User mobility
 - ▶ Availability
 - ▶ Diskless workstations

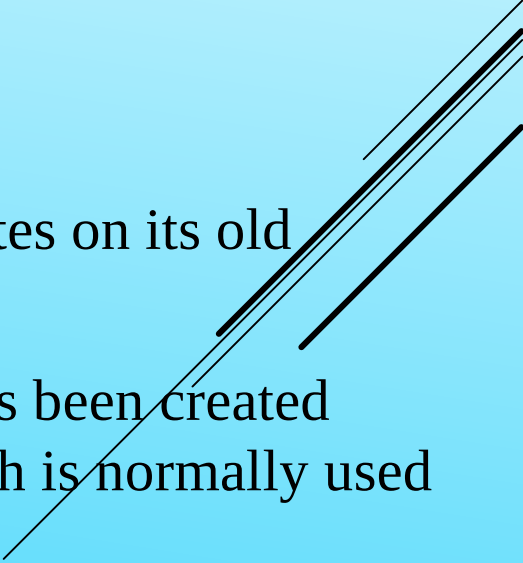
Services provided by DFS

1. **Storage service.** It deals with the allocation and management of space on a secondary storage device that is used for storage of files in the file system.
2. **True file service.** It is concerned with the operations on individual files, such as operations for accessing and modifying the data in files and for creating and deleting files.
3. **Name service.** It provides a mapping between text names for files and references to files, that is, file IDs

Desirable Features of a DFS

- Transparency:
 - Structure transparency
 - Access transparency
 - Naming transparency
 - Replication transparency
- User Mobility
- Performance
- Simplicity and ease of use
- Scalability
- High availability
- Data integrity
- Security
- Heterogeneity

File Models

- Based on structure and modifiability two models:
 - Unstructured and Structured Files
 - **Unstructured files:** a file is an unstructured sequence of data
 - **Structured files:** a file appears to the file server as an ordered sequence of records. Two types:
 - Files with nonindexed records
 - Files with indexed records
 - Mutable and Immutable Files
 - **Mutable:** an update performed on a file overwrites on its old contents.
 - **Immutable:** a file cannot be modified once it has been created except to be deleted. The file versioning approach is normally used to implement file updates
- 

File Accessing models

- The file-accessing model of a distributed file system depends on two factors -
 - Accessing remote files
 - Remote Service Model
 - Data-Caching Model
 - Unit of data access.
 - File-level transfer model
 - Block-level transfer model
 - Byte-level transfer model
 - Record-level transfer model

File Accessing models

➤ Accessing remote files

- **Remote Service Model:** the processing of the client's request is performed at the server's node.
 - Client sends request, server performs and send back the result to client.
 - the data packing and communication overheads per message can be significant
- **Data-Caching Model:** the data needed to satisfy the client's access request is copied from the server's node to the client's node and is cached there.
 - reduces network traffic
 - there is a possibility that data of a particular file may simultaneously be available in different caches so that consistency problem
 - increased performance and greater system scalability because it reduces network traffic, contention for the network
- Hybrid Models: LOCUS, NFS and Sprite

File Accessing models

➤ Unit Data Transfer (DT)

When we use the data-caching model, an important design issue is to decide the unit of DT

➤ **File-level transfer model** (Amoeba, CFS, AFS)

- more efficient, better scalability
- disk access routines on the servers can be better optimized
- offers a degree of intrinsic resiliency
- simplifies the task of supporting heterogeneous workstations
- requires sufficient storage space on the client's node

➤ **Block-level transfer model** (DFS, LOCUS, NFS and Sprite)

- Does not require large storage space & used in systems having diskless workstations
- more network traffic and more network protocol overhead

➤ **Byte-level transfer model** (Cambridge File Server uses this model)

- maximum flexibility and difficulty in cache management

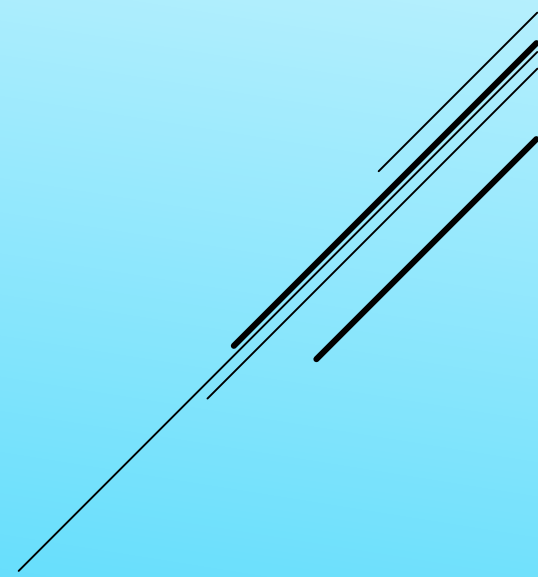
➤ **Record-level transfer model** (Research Storage System: RSS)

- file contents are structured in the form of records

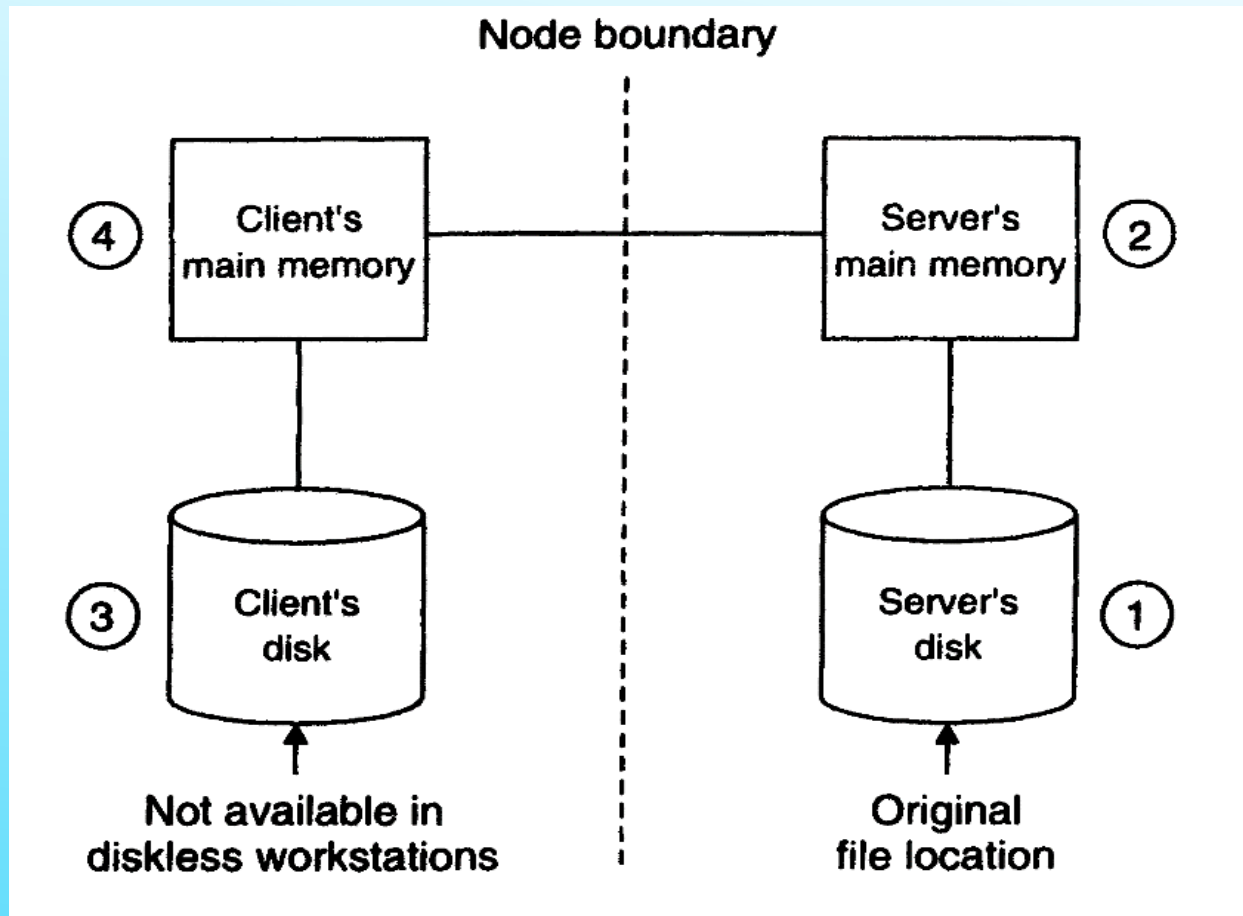
File-Caching Schemes

File caching has been implemented in several file systems for centralized time-sharing systems to **improve file I/O performance**

- A file-caching scheme for a distributed file system should address the following key decisions:
 1. Cache location
 2. Modification propagation
 3. Cache validation



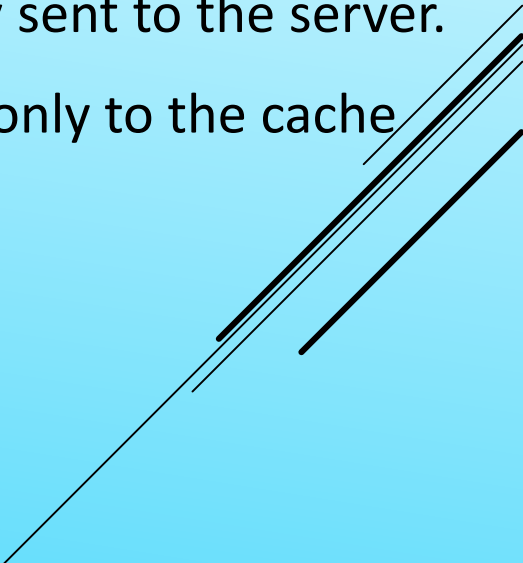
File-Caching Schemes: 1. Cache location



File-Caching Schemes: 1. Cache location

Cache location	Access cost on cache hit	Advantages and Disadvantages
Server's main memory	One network access	1. Easy to implement 2. Totally transparent to the clients 3. Easy to keep the original file & cached data consistent 4. Easy to support UNIX like file-sharing semantics 1. It does not eliminate the network access cost 2. Does not contribute to the scalability and reliability
Clients disk	One disk access	1. Reliability against crashes 2. Large storage capacity 3. Suitable for supporting disconnected operation 4. Contributes to scalability and reliability 1. Does not work to support diskless workstations 2. The access time is still considerably large
Clients main memory	--	1. Maximum performance gain memory 2. Permits workstations to be diskless 3. Contributes to scalability and reliability

File-Caching Schemes: 2. Modification Propagation

- The caches of all the nodes must contain exactly the same copies of the file data (the caches are consistent).
 - When to propagate modifications made to a cached data to the corresponding file server (**Modification Propagation schemes**). Two approaches used:
 1. Write-through Scheme: new value is immediately sent to the server.
 2. Delayed-Write Scheme: the new value is written only to the cache
 - ▶ Write on ejection from cache
 - ▶ Periodic write.
 - ▶ Write on close
- 

File-Caching Schemes: 2. Modification Propagation

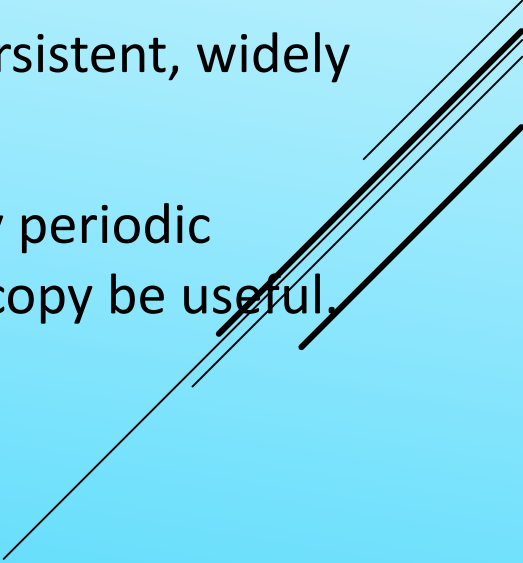
- **Modification Propagation schemes: Delayed write Policy**
- The delayed-write policy helps in performance improvement for write accesses due to the following reasons:
 - Write accesses complete more quickly.
 - Modified data may be deleted before sending them to the server.
 - Gathering of all file updates and sending them together to the server is more efficient than sending each update separately.
- Drawbacks:
 - Suffer from reliability problems, since modifications not yet sent to the server from a client's cache will be lost if the client crashes.
 - Delaying the propagation of modifications to the server results in fuzzier file-sharing semantics, because when another process reads the file, what it gets depends on the timing.

File-Caching Schemes: 3. Cache Validation Schemes

- It is necessary to verify the data cached at a client node is consistent with the master copy. If not, the cached data must be invalidated and the updated version of the data must be fetched again from the server
 - Two Approaches:
 - Client-initiated approach
 - Checking before every access
 - Periodic checking
 - Check on file open
 - Server-initiated approach
 - It violates the traditional client-server model
 - It requires that file servers be stateful
 - A check-on-open, client-initiated cache validation approach must still be used along with the server-initiated approach
- 
- A series of parallel diagonal lines in the bottom right corner of the slide, consisting of a thin black line and a thicker grey line.

File Replication

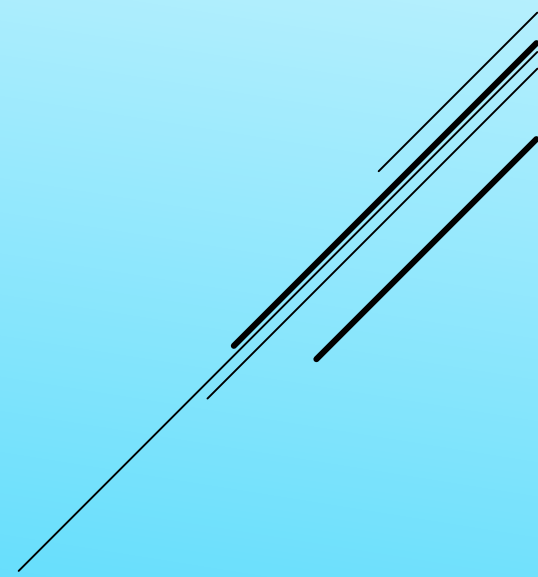
Difference between Replication and Caching

1. A replica is associated with a server, whereas a cached copy is normally associated with a client.
 2. The existence of a cached copy is primarily dependent on the locality in file access patterns, whereas the existence of a replica normally depends on availability and performance requirements.
 3. As compared to a cached copy, a replica is more persistent, widely known, secure, available, complete, and accurate.
 4. A cached copy is dependent upon a replica. Only by periodic revalidation with respect to a replica can a cached copy be useful.
- 
- A series of three parallel diagonal lines in the bottom right corner of the slide, extending from the bottom right towards the center.

File Replication

Advantages of Replication

1. Increased availability.
2. Increased reliability.
3. Improved response time.
4. Reduced network traffic.
5. Improved system throughput.
6. Better scalability.



File Replication: Replication Transparency (issues)

1. Naming of Replicas:

- ▶ **For immutable objects:** Assignment of a single identifier to all replicas of an object seems reasonable.
- ▶ **For mutable objects:** different copies of a replicated object may not be the same (consistent) at a particular instance of time. In this case, if the same identifier is used for all replicas of the object, the kernel cannot decide which replica is the most up-to-date one.
 - ▶ naming system is responsible to map a user-supplied identifier into the appropriate replica of a mutable object.

2. Replication Control: determining the number and locations of replicas of a replicated file

- ▶ **Explicit replication:** users are given the flexibility to control the entire replication process
- ▶ **Implicit/Lazy replication:** The entire replication process is automatically controlled by the system without users' knowledge

File Replication: Multi-copy Update Problem

1. **Read-Only Replication:** allows the replication of only immutable files
2. **Read-Any-Write-All Protocol:** a read operation on a replicated file is performed by reading any copy of the file and a write operation by writing to **all copies of the file**. Some form of locking has to be used to carry out a write operation.
 - ▶ a write operation cannot be performed if any of the servers having a copy of the replicated file is down at the time of the write operation.
3. **Available-Copies Protocol:** a read operation is performed by reading any available copy, but a write operation is performed by writing **to all available copies**.
 - ▶ when a server recovers after a failure, it brings itself up to date by copying from other servers before accepting any user request
4. **Primary-Copy Protocol:** for each replicated file, one copy is designated as the primary copy and all the others are secondary copies. Read operations can be performed using any copy, primary or secondary. But, all write operations are directly performed only on the primary copy

File Replication: Multi-copy Update Problem

- 5. Quorum-Based Protocols:** used for handling the network partition problem and can increase the availability of write operations at the expense of read operations.
- ▶ A quorum-based protocol works as follows.
 - ▶ Suppose that there are a total of n copies of a replicated file F . To read the file, a minimum r copies of F have to be consulted. This set of r copies is called a read quorum.
 - ▶ Similarly, to perform a write operation on the file, a minimum w copies of F have to be written. This set of w copies is called a write quorum.
 - ▶ The restriction on the choice of the values of r and w : $r + w > n$.
 - ▶ This restriction guarantees that there is a nonnull intersection between every read quorum and every write quorum.

File Replication: Multi-copy Update Problem

- ▶ Quorum based protocol does not require that write operations be executed on all copies of a replicated file.
- ▶ **Requirement:** to identify a current (up-to-date) copy in a quorum.
 - ▶ This is achieved by associating a version number attribute with each copy. It is updated every time the copy is modified. A copy with the largest version number in a quorum is a current copy.
 - ▶ The new version number assigned to each copy is one more than the version number associated with the current copy used for handling the network partition problem and can increase the availability of write operations at the expense of read operations.
- ▶ **A read is executed as follows:**
 1. Retrieve a read quorum (any r copies) of F .
 2. Of the r copies retrieved, select the copy with the largest version number.
 3. Perform the read operation on the selected copy.

File Replication: Multi-copy Update Problem

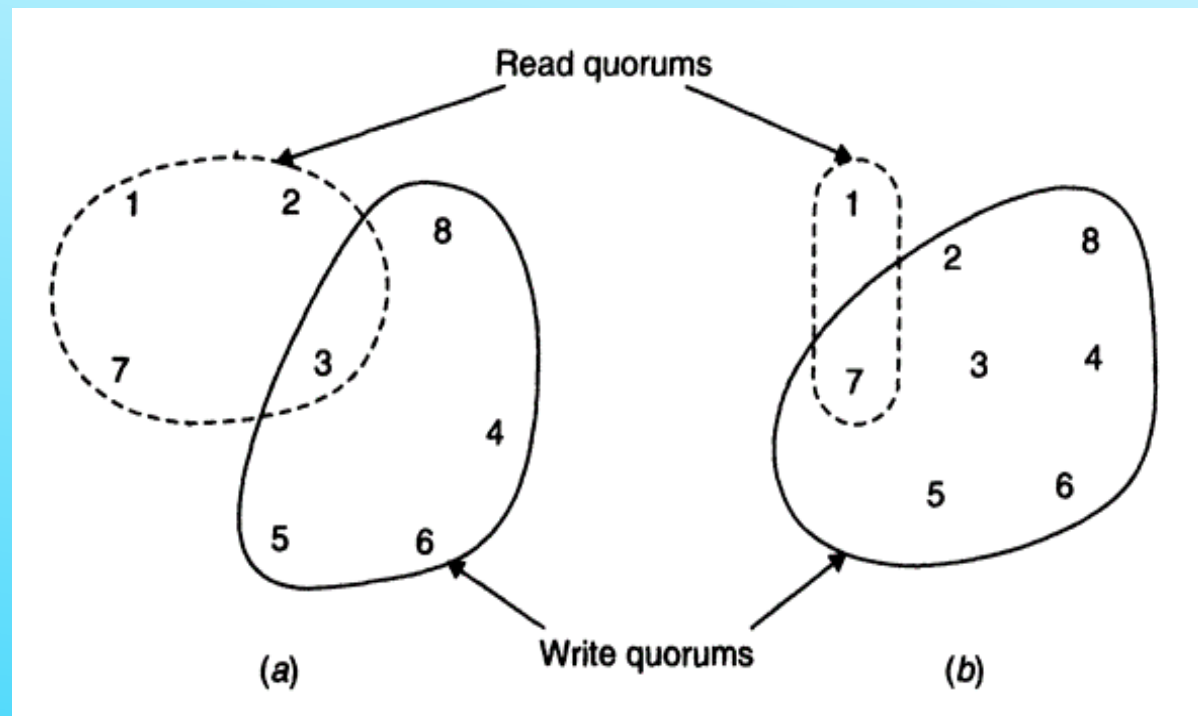
► A write is executed as follows:

1. Retrieve a write quorum (any w copies) of F .
2. Of the w copies retrieved, get the ver-no. of the copy with the largest ver-no.
3. Increment the version number.
4. Write the new value and the new version number to all the w copies of the write quorum.

Examples of quorum consensus algorithm:

(a) $n = 8$; $r = 4$; $w = 5$

(b) $n = 8$; $r = 2$; $w = 7$



File Replication: Multi-copy Update Problem

Some special algorithms can be derived from quorum protocol. They are:

- ▶ **Read-any-write-all protocol.** It is a special case of the generalized quorum protocol with $r=1$ and $w = n$. This protocol is suitable for use when the ratio of read to write operations is large.
- ▶ **Read-all-write-any protocol.** Where $r=n$ and $w= 1$. This protocol may be used in those cases where the ratio of write to read operations is large.
- ▶ **Majority-consensus protocol.** Here, the sizes of both the read quorum and the write quorum are made either equal or nearly equal. This protocol is commonly used in those cases for which the ratio of read to write operations is nearly 1.
- ▶ **Consensus with weighted voting.** The varying "importance" can be given to different copies of a replicated file by assigning each copy with some number of votes.